

FORMATION EMBARQUÉE

TD 06 Autres Langages

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 06 — Autres langages/outils : énoncés et corrigés

Exercice 1 — Lire l'assembleur produit par GCC

Énoncé. Compile cette fonction avec `-O0` puis `-O2`, désassemble, explique deux optimisations observées.

```
#include <stdint.h>
uint32_t somme(const uint32_t *t, uint32_t n) {
    uint32_t s = 0;
    for (uint32_t i = 0; i < n; i++)
        s += t[i] * 2;
    return s;
}
```

```
gcc -O0 -S somme.c -o somme_00.s
gcc -O2 -S somme.c -o somme_02.s
# ou, en croisé : arm-none-eabi-gcc -mcpu=cortex-m4 -O2 -S somme.c
# ou en ligne : https://godbolt.org (le plus confortable)
```

Corrigé (observations attendues)

1. **Variables en registres** : en `-O0`, `s` et `i` vivent **en pile** — chaque tour fait des chargements/rangements mémoire (`ldr` / `str` autour de chaque opération). En `-O2`, tout tient dans des registres : le corps de boucle se réduit à ~3 instructions. C'est la raison pour laquelle on ne mesure JAMAIS de performance en `-O0`.
2. **Multiplication remplacée** : `* 2` devient un décalage (`lsl #1`) ou s'intègre au mode d'adressage — le compilateur fait tout seul les « astuces » bit à bit du module 01.
3. (Bonus) En `-O2` sur x86, GCC **vectorise** souvent (instructions SIMD traitant 4 éléments par tour) et/ou déroule la boucle ; sur Cortex-M, il fusionne le test de fin avec la soustraction.
4. (Bonus, à retenir) : si `s` était `volatile`, quasi toutes ces optimisations disparaîtraient — relie ça au rôle exact de `volatile`.

Exercice 2 — Makefile pour un projet 3 fichiers

Énoncé. `main.c`, `capteur.c`, `affichage.c` (+ leurs `.h`) : cibles `all`, `clean`, reconstruction minimale.

Corrigé

```
CC      := gcc
CFLAGS  := -Wall -Wextra -O2 -MMD -MP
SRCS    := main.c capteur.c affichage.c
OBJS     := $(SRCS:.c=.o)
DEPS     := $(OBJS:.o=.d)
CIBLE   := app

all: $(CIBLE)

$(CIBLE): $(OBJS)
    $(CC) $(CFLAGS) $^ -o $@

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJS) $(DEPS) $(CIBLE)

-include $(DEPS)

.PHONY: all clean
```

Points de correction : - La **reconstruction minimale** vient de deux mécanismes : la règle générique `%.o: %.c` (make compare les dates `.c/.o`) et les fichiers de dépendances générés par `-MMD -MP` puis inclus par `-include $(DEPS)` — sans eux, modifier `capteur.h` ne recompile pas `main.c` qui l'inclut : c'est LE piège classique. - `.PHONY : all` et `clean` ne sont pas des fichiers ; sans cette ligne, un fichier nommé `clean` casserait la cible. - `$@` = la cible, `$<` = la première dépendance, `$^` = toutes. - Test : `make` (tout compile) → `touch capteur.h` → `make` (main.o et capteur.o se recompilent, pas affichage.o) → `make` (rien à faire).

Exercice 3 — Deux tâches FreeRTOS sur ESP32

Énoncé. Une tâche clignote une LED à 2 Hz, l'autre imprime un compteur chaque seconde ; communication par queue.

Corrigé (IDE Arduino, cible ESP32 — FreeRTOS est déjà là)

```
#include <Arduino.h>

QueueHandle_t file_compteur;

void tacheBlink(void *param) {
    pinMode(2, OUTPUT);
    for (;;) {
        digitalWrite(2, !digitalRead(2));
        // vTaskDelay REND le CPU : l'autre tâche (et le Wi-Fi !) tournent.
        // Un delay() Arduino ferait pareil ici, mais prends l'habitude RTOS.
        vTaskDelay(pdMS_TO_TICKS(250));    // 250 ms → 2 Hz
    }
}

void tacheCompteur(void *param) {
    uint32_t n = 0;
    for (;;) {
        n++;
        xQueueSend(file_compteur, &n, 0);    // 0 : n'attend pas si pleine
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void setup() {
    Serial.begin(115200);
    file_compteur = xQueueCreate(8, sizeof(uint32_t));

    // (fonction, nom, pile en MOTS, param, priorité, handle, cœur)
    xTaskCreatePinnedToCore(tacheBlink, "blink", 2048, NULL, 1, NULL, 1);
    xTaskCreatePinnedToCore(tacheCompteur, "cpt", 2048, NULL, 1, NULL, 1);
}

void loop() {
    uint32_t n;
    // loop() sert de tâche "affichage" : elle BLOQUE sur la queue
    if (xQueueReceive(file_compteur, &n, portMAX_DELAY) == pdTRUE)
        Serial.printf("compteur = %lu\n", (unsigned long) n);
}
```

Points de correction : - Le compteur passe par **la queue**, pas par une variable globale : pas de **volatile**, pas de section critique, transfert par copie — c'est tout l'intérêt. - **vTaskDelay** (bloquant-coopératif) et non une boucle d'attente active qui affamerait les tâches de priorité inférieure. - Taille de pile : 2048 est confortable ; en cas de plantages aléatoires, premier réflexe RTOS = **uxTaskGetStackHighWaterMark()**.

Exercice 4 — Station météo en MicroPython (comparaison)

Corrigé (Pico + DHT22 + affichage console, complet)

```
from machine import Pin
import dht, time

capteur = dht.DHT22(Pin(15))
led = Pin(25, Pin.OUT)

SEUIL_ALERTE = 28.0

while True:
    try:
        capteur.measure()
        t = capteur.temperature()
        h = capteur.humidity()
        print("T = {:.1f} C  H = {:.0f} %".format(t, h))
        led.value(1 if t > SEUIL_ALERTE else 0)
    except OSError:
        print("capteur muet, nouvelle tentative") # panne = cas géré, pas crash
    time.sleep(2)
```

Analyse attendue (le fond de l'exercice) : la version MicroPython tient en 15 lignes et se met au point en direct via le REPL — contre une heure et des bibliothèques en C++. En échange : ~100× plus lent, RAM consommée par l'interpréteur, latences avec le ramasse-miettes → parfait pour un prototype ou un capteur lent, disqualifié pour du temps réel serré. Conclusion à formuler : **le langage se choisit par les contraintes, pas par le confort.**

Auto-évaluation

Sans notes : expliquer `-O0` vs `-O2` et pourquoi `volatile` bride l'optimiseur ; écrire de tête la règle générique `%.o: %.c` avec `$< / $@` ; justifier une queue RTOS face à une globale partagée ; donner deux critères qui disqualifient MicroPython pour une application donnée.